

HabitFlow

Habit Tracking

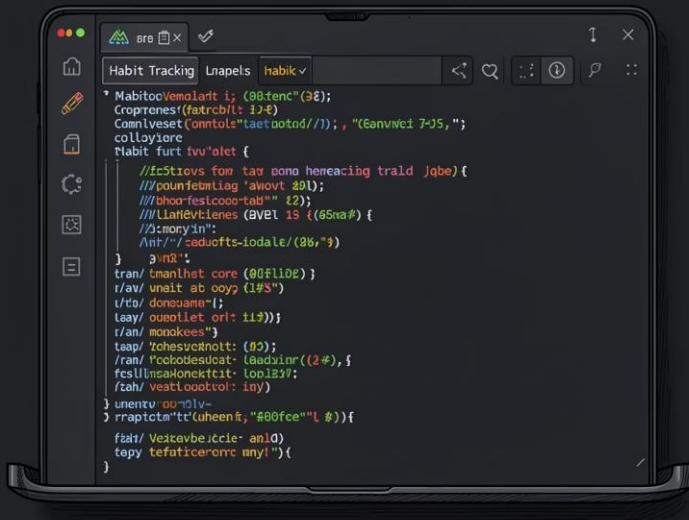
Application

Phase 2 — Design & Implementation

Object Oriented & Functional Programming with Python



What Is HabitFlow?



Project Overview:

HabitFlow is a Python-based habit tracking application that helps users build and maintain positive habits.

Key Features:

- Create daily and weekly habits
- Track habit completion
- Store habit data using SQLite
- Analyse habit streaks
- Perform automated testing with unittest

Project Goals:

- Apply Object-Oriented Programming principles
- Use Functional Programming for analytics
- Persist data using a relational database
- Build a modular and maintainable application

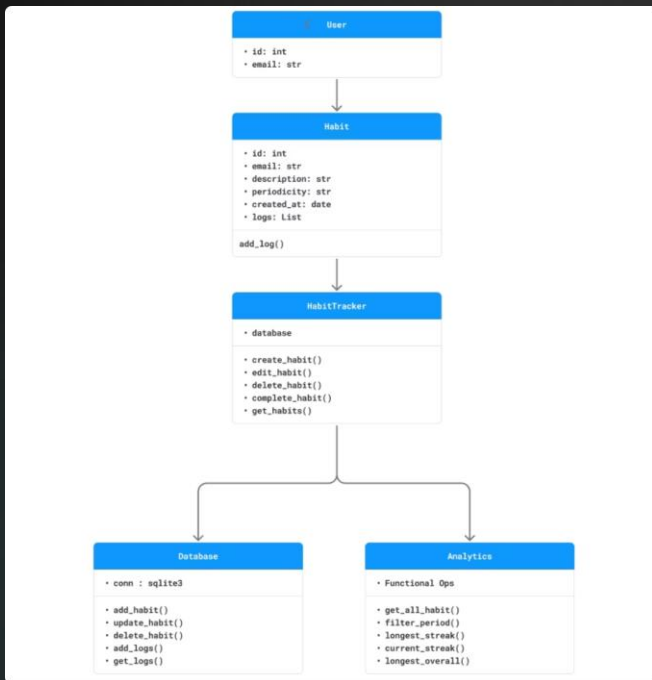
Daily & Weekly Habits

SQLite Persistence

Analytics & Streaks

Unit Testing

UML Class Diagram & Key Design Decisions



Design Decisions:

- Each class has a single responsibility
- HabitLog represents a single habit completion event
- Analytics functions are separated from data storage
- SQLite provides persistent data management

Habit

Encapsulates name, description, and periodicity

HabitLog

Tracks individual completion timestamps

Analytics

Isolated from storage — pure computation

Database

SQLite layer abstracted behind a clean API

System Architecture

HabitFlow uses a **modular file-per-module architecture**. Each Python file owns a distinct concern, making the system easy to navigate, test, and extend.



main.py

Controls program flow and CLI interaction



database.py

Handles all SQLite read/write operations



models.py

Defines Habit and HabitLog classes



analytics.py

Calculates streaks and progress metrics



predefined.py

Loads example habits for onboarding

Database Design

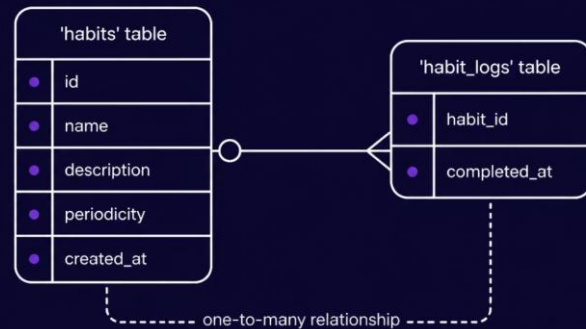
SQLite Schema

habits table

- `id` — Primary key
- `name` — Habit title
- `description` — Optional detail
- `periodicity` — daily or weekly
- `created_at` — Timestamp

habit_logs table

- `habit_id` — Foreign key
- `completed_at` — Completion timestamp



Why SQLite? SQLite was selected instead of JSON because it provides structured storage and supports relationships between habits and habit completion records..

Analytics Module



Functional Programming Paradigm Validation (analytics.py)

The analytics engine is designed explicitly around the Functional Programming (FP) model to guarantee mathematical safety and avoid application side effects.

1. Pure Computational Logic:

Functions like `get_longest_streak(habit)` and `get_current_streak(habit)` operate strictly as pure functions. They do not alter database records, nor do they touch global scopes. They accept data types, evaluate them, and return a result.

2. State Isolation via Transformations:

Instead of mutating object properties directly, the analytics engine uses data stream transformations:

- `logs = sorted([l.completed_at for l in habit.logs])`

This expression extracts tracking logs into isolated list structures before calculating streaks, preserving the immutability of the core application objects.

3. Deterministic Streak Calculations:

The structural progression is determined using explicit datetime math (`is_next_period()`), ensuring predictable metrics under verification testing.

1

`get_longest_streak()`

Scans the full completion history and returns the longest consecutive sequence, accounting for daily or weekly periodicity.

2

`get_current_streak()`

Calculates the active streak counting backward from today. Resets to zero if a period is missed.

Unit Testing

All core functionality is validated using Python's built-in `unittest` framework, ensuring reliability across habit creation, logging, and streak calculation.



Habit Creation

Verifies habits are instantiated and persisted with correct attributes.



Current Streak

Tests active streak calculation across daily and weekly habits.



Habit Completion

Confirms completion logs are recorded with accurate timestamps.



Longest Streak

Validates the longest consecutive sequence is correctly identified.

What Was Delivered

✓ Completed Components

- Object-Oriented class design
- SQLite database with two-table schema
- Analytics module with streak logic
- Automated unit test suite
- Modular, maintainable architecture

Future Roadmap

1 Graphical Interface

Migrate from CLI to a GUI (e.g., Tkinter or a web frontend)

2 Reminder System

Schedule notifications for pending habits

3 Export Reports

Generate progress summaries as CSV or PDF

Thank You

HabitFlow demonstrates a complete Python application lifecycle — from object-oriented design through database modeling, analytics, and automated testing.

Design

UML & OOP principles

Build

Python & SQLite

Validate

unittest coverage

